

```
# Initialize Swarm
docker swarm init

# Deploy stack
docker stack deploy -c docker-compose.yml <stack-name>

# List services
docker service ls

# Scale service
docker service scale <stack-name>_<service-name>=<replicas>

# See service tasks
docker service ps <service-name>

# Remove stack
docker stack rm <stack-name>

# Leave Swarm (if needed)
docker swarm leave --force
```

Experiment 12: Study and Analyse Container Orchestration using Kubernetes Objective

Learn why Kubernetes is used, its basic concepts, and how to deploy, scale, and fix apps using Kubernetes commands.

Why Kubernetes over Docker Swarm?

Reason	Explanation
Industry standard	Most companies use Kubernetes
Powerful scheduling	Automatically decides where to run your app
Large ecosystem	Many tools and plugins available
Cloud-native support	Works on AWS, Google Cloud, Azure, etc.

Core Kubernetes Concepts (Simple Explanation)

Docker Concept	Kubernetes Equivalent	What it means
Container	Pod	A pod is a group of one or more containers. Smallest unit in K8s.
Compose service	Deployment	Describes how your app should run (e.g., 2 copies, which image to use)
Load	Service	Exposes your app to the outside world or other

balancing			
		pods	
Docker	Kubernetes		What it means
Concept	Equivalent		
		Ensures a certain number of pod copies are always	
Scaling			
		ReplicaS et running	

Hands-On Lab (Using k3d or Minikube)

Note: We assume you already have kubectl and a cluster (k3d or Minikube) installed. If not, ask your instructor.

Task 1: Create a Deployment

A deployment tells Kubernetes:

- Which container image to use (e.g., WordPress)
- How many copies (replicas) to run
- How to identify the pods (labels)

Step 1: Create a file `wordpress-deployment.yaml`

```
# wordpress-deployment.yaml
apiVersion: apps/v1           # Which Kubernetes API to use
kind: Deployment              # Type of resource
metadata:
  name: wordpress             # Name of this deployment
spec:
  replicas: 2                  # Run 2 identical pods
```

```

selector:                # Pods with this label belong to this
  matchLabels:            deployment
  app: wordpress          # Template for the pods
template:
  metadata:
  labels:                 # Label applied to each pod
  app: wordpress
  spec:
  containers:
    - name:
wordpress
  image: wordpress:latest # Docker image    ports:
    - containerPort: 80    # Port inside the container

```

Step 2: Apply the deployment

```
kubectl apply -f wordpress-deployment.yaml
```

What happens?

Kubernetes creates 2 pods running WordPress.

```

aryanraj@aryanraj:~/exp12$ nano wordpress-deployment.yaml
aryanraj@aryanraj:~/exp12$ kubectl apply -f wordpress-deployment.yaml
deployment.apps/wordpress created
aryanraj@aryanraj:~/exp12$ █

```

Task 2: Expose the Deployment as a Service

Pods are temporary (they can be deleted or recreated).

A Service gives them a fixed IP and exposes them to the outside.

Step 1: Create a file **wordpress-service.yaml**

```

# wordpress-service.yaml
apiVersion: v1 kind:
Service metadata:
  name: wordpress-service spec:
  type: NodePort #Exposes service on a port of each node (VM) selector:
  app: wordpress #Send traffic to pods with this label ports:
    - port: 80 #Service port    targetPort: 80
      #Pod port
      nodePort: 30007    #External port (range: 30000–32767)

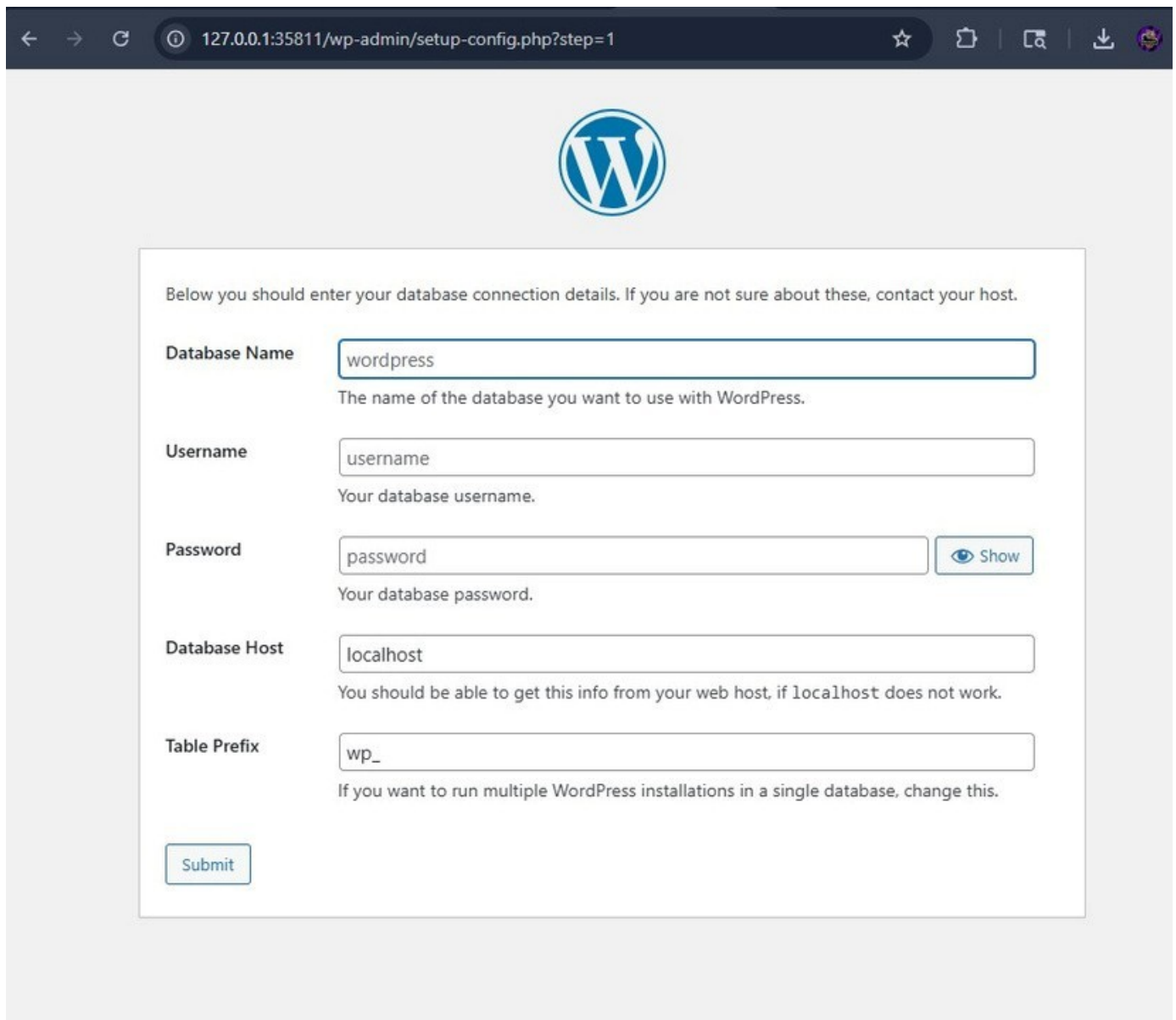
```

Step 2: Apply the service

```
kubectl apply -f wordpress-service.yaml
```


How to find node-ip in k3d/minikube?

- Minikube: `minikube ip`
- k3d: Usually localhost



The screenshot shows a web browser window with the address bar displaying `127.0.0.1:35811/wp-admin/setup-config.php?step=1`. The page features the WordPress logo at the top center. Below the logo, a text instruction reads: "Below you should enter your database connection details. If you are not sure about these, contact your host." The form contains five input fields, each with a label and a description:

- Database Name:** The input field contains "wordpress". The description below it says: "The name of the database you want to use with WordPress."
- Username:** The input field contains "username". The description below it says: "Your database username."
- Password:** The input field contains "password". To the right of the field is a "Show" button with an eye icon. The description below it says: "Your database password."
- Database Host:** The input field contains "localhost". The description below it says: "You should be able to get this info from your web host, if localhost does not work."
- Table Prefix:** The input field contains "wp_". The description below it says: "If you want to run multiple WordPress installations in a single database, change this."

At the bottom left of the form is a "Submit" button.

Task 4: Scale the Deployment

Increase the number of pods from 2 to 4: `kubectl scale`

```
deployment wordpress --replicas=4
```

Verify:

```
kubectl get pods
```

You should now see 4 running pods.

Why scale? More traffic → more copies → better performance.

```
aryanraj@aryanraj:~/exp12$ kubectl scale deployment wordpress --replicas=4
deployment.apps/wordpress scaled

aryanraj@aryanraj:~/exp12$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
wordpress-6698dd7d66-4tk6f	1/1	Running	0	15m
wordpress-6698dd7d66-7sr18	1/1	Running	0	7s
wordpress-6698dd7d66-drk2m	1/1	Running	0	15m
wordpress-6698dd7d66-hf5t8	0/1	ContainerCreating	0	7s

```
aryanraj@aryanraj:~/exp12$
```

Task 5: Self-Healing Demonstration

Kubernetes automatically replaces failed pods.

Delete one pod manually:

```
# First, get pod names kubectl get pods
```

```
# Delete one (replace <pod-name>) kubectl
```

delete pod <pod-name> Now check pods again:

```
kubectl get pods
```

You will still see 4 pods — the deleted one was automatically recreated.

Why? The deployment ensures the desired number (4) is always running.

```
aryanraj@aryanraj:~/exp12$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
wordpress-6698dd7d66-4tk6f	1/1	Running	0	15m
wordpress-6698dd7d66-7sr18	1/1	Running	0	29s
wordpress-6698dd7d66-drk2m	1/1	Running	0	15m
wordpress-6698dd7d66-hf5t8	1/1	Running	0	29s

```
aryanraj@aryanraj:~/exp12$ kubectl delete pod ^C
aryanraj@aryanraj:~/exp12$ kubectl get pods wordpress-6698dd7d66-7sr18
```

NAME	READY	STATUS	RESTARTS	AGE
wordpress-6698dd7d66-7sr18	1/1	Running	0	60s

```
aryanraj@aryanraj:~/exp12$ kubectl delete pod wordpress-6698dd7d66-7sr18
pod "wordpress-6698dd7d66-7sr18" deleted from default namespace
aryanraj@aryanraj:~/exp12$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
wordpress-6698dd7d66-2k8xk	1/1	Running	0	8s
wordpress-6698dd7d66-4tk6f	1/1	Running	0	16m
wordpress-6698dd7d66-drk2m	1/1	Running	0	16m
wordpress-6698dd7d66-hf5t8	1/1	Running	0	97s

```
aryanraj@aryanraj:~/exp12$
```

PART C – Swarm vs Kubernetes (Simple Comparison)

Feature	Docker Swarm	Kubernetes
Setup	Very easy	More complex
Scaling	Basic	Advanced (auto-scaling)
Ecosystem	Small	Huge (monitoring, logging)
Industry use	Rare	Standard

Verdict: Learn Kubernetes — it's what companies use.

PART D – Advanced Lab: Real Cluster with kubeadm

Question: Can we set up a real Kubernetes cluster?

Yes — using **kubeadm**.

This is closer to a production setup.

Lab Requirements

- 2 or 3 virtual machines (e.g., VirtualBox, VMware)
- Ubuntu 22.04 or 24.04
- Each VM: 2+ CPU, 2+ GB RAM

High-Level Steps

Step 1: Install kubeadm, kubelet, kubectl on all nodes

```
# Update system
sudo apt update

# Install required packages
sudo apt install -y apt-transport-https ca-certificates curl

# Add Kubernetes signing key
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.29/deb/Release.key | sudo gpg --dearmor -o

# Add Kubernetes repository
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.29/deb/

# Install kubeadm, kubelet, kubectl
sudo apt update
sudo apt install -y kubeadm kubelet kubectl

# Hold versions to prevent auto-update
sudo apt-mark hold kubeadm kubelet kubectl
```

Step 2: Initialize the control plane (master node) Run

only on the master node: `sudo kubeadm init`

This command:

- Sets up the control plane
- Generates a token for worker nodes to join Takes 2-3
- minutes

Step 3: Set up kubeconfig (to use kubectl)

```
# Create .kube directory for your user mkdir -p
$HOME/.kube

# Copy admin config sudo cp /etc/kubernetes/admin.conf
$HOME/.kube/config

# Fix permissions
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Now you can run `kubectl get nodes` to see the master.

Step 4: Install a network plugin (required for pods to communicate)

```
kubectl apply -f https://docs.projectcalico.org/manifests/calico.yaml
```

Wait 1-2 minutes for Calico to start.

Step 5: Join worker nodes

On the master, after kubeadm init, you'll see a join command like:

```
kubeadm join 192.168.1.100:6443 --token abcdef.0123456789abcdef \
--discovery-token-ca-cert-hash sha256:...
```

Run that exact command on each worker node.

Step 6: Verify the cluster On the

master node: `kubectl get nodes`

Expected output:

NAME	STATUS	ROLES	AGE	VERSION	master	Ready
	control-plane		5m	v1.29.0		
worker1	Ready	<none>		2m	v1.29.0	
worker2	Ready	<none>		2m	v1.29.0	

Teaching Insight: When to Use Which Tool

Tool	Best for k3d	Quick learning on your laptop
Minikube	Single-node cluster testing	
kubeadm	Real, production-style cluster	

Summary of Commands (Cheat Sheet)

Goal	Command
Apply a YAML file	<code>kubectl apply -f file.yaml</code>
See all pods	<code>kubectl get pods</code>
See all services	<code>kubectl get svc</code>
Scale a deployment	<code>kubectl scale deployment <name> --replicas=N</code>
Delete a pod	<code>kubectl delete pod <pod-name></code>
See all nodes	<code>kubectl get nodes</code>

End of Experiment

You have now:

- Understood basic Kubernetes concepts
- Deployed WordPress using a Deployment and Service
- Scaled and tested self-healing
- Learned how to build a real cluster with kubeadm

Next step: Try deploying your own custom app (e.g., a simple Node.js or Python app).